Fall 2026 CSC 582, Assignment #2

Modeling an Unfamiliar System Through Abstraction: The Options Market

Read first.

- This is **individual** work. **You write no code that simulates the market.** The only code you write is Mermaid script, which generates your UML diagrams.
- You model an unfamiliar domain, described in `OPTIONS-MARKETS.md` (§3). You draw one class diagram and two sequence diagrams in Mermaid, and you *also* explain your model in a written rationale. The writing does not replace any diagram.
- You have **two weeks** for the take-home.
- All marks come from questions in a **monitored in-class session**: Part A assesses your take-home (75%), and Part B asks you to extend it (25%).
- You may use AI assistants during the take-home if you report them (§8), but not in the in-class session (§9).

Sections *(each links to its section)*

1 Deliverables and grading
2 Introduction: this assignment and Asg1
3 The domain, and the materials you are given
4 What you have to model
5 The rule, and how it is marked
6 What you submit: Mermaid, and prose
7 The design rationale
8 AI assistant policy
9 The in-class session
10 Submission

1 Deliverables and grading

Component, and what is looked at	Weight
Part A — the take-home (75%), assessed by in-class questions and your bundle	
1. Domain document — your marks (varies, fixed, impossible), committed while reading (§3)	4%
2. Coverage — your diagrams have every item in §4, with correct relationships	14%
3. Separation of variation — no operation checks kinds (§5); new kinds change few classes	14%
4. Contract life — open, exercised, expired, and what each state allows (§4)	9%
5. Rationale — §7 answered for your diagrams; $\geq$2 rejected alternatives; misfits	14%
6. Mermaid that parses — all three diagrams render, with the detail §6 asks for	7%
7. Commit history — $\geq$20 commits on $\geq$7 different days, each with a real message	8%
8. `AI_USAGE.md` — kept up to date, with every important use reported (§8)	5%
Part B — the extension (25%), in-class questions about a design you make in the in-class session	
9. The design, in prose — what it adds, changes, and keeps, and why (§9)	17%
10. A Mermaid class diagram of the extension, and any misfits it shows	8%

You are not expected to be perfect. **Asg2 grades are curved** to a class average of 75. Your grade depends on how carefully and thoroughly you carry out each part, compared with your classmates.

2 Introduction: this assignment and Asg1

In Assignment #1 you *implemented* a small system. It was fully specified, with one axis of variation, and every design decision had a clear answer from the start.

This assignment covers the other half of design: the work that comes *before* implementation. You get a real domain, listed options markets, described in a short document of plain prose. The description is incomplete on purpose. You produce an **object model**: a UML class diagram, two sequence diagrams, and a written design rationale. As a software engineer, you will often work in a domain you do not know well. Your success will depend on how well you learn it, using your computer science skills to analyze it. This assignment tests exactly that: **how well each of you, individually, can deal with the complexity of a new domain and represent it in a design with objects**. Choose a technique for each decision separately:

- **inheritance**, when one kind of thing really *is* a kind of another;
- **interfaces or mixins**, when several unrelated kinds of thing share one capability;
- **multiple inheritance**, when one thing really is two things at once.

If you use the same technique everywhere, you have not made a choice; your rationale must show your choices. A call and a put share almost everything and differ in one thing: which way the shares and the cash move (doc §4.2). Your model must express that difference in one place, without repeating everything else.

3 The domain, and the materials you are given

Everything you need is in the public package repository. Start with its `README.md`, and work through its `WALKTHROUGH.md` on the first day:

`https://github.com/behollister/CSC582-Asg02-Fa26-Dist`

Your model is based on one document: `OPTIONS-MARKETS.md`, at the top of the repository. In this handout, doc §5 means section 5 of that document; a § without "doc" is a link to a section of this handout.

- `OPTIONS-MARKETS.md` — the domain description, with worked examples and real calculations. doc §0 explains how to read it as a modeler: mark *what varies*, *what is fixed*, and *what cannot happen*. Your marked-up copy is rubric item 1.
- **Black–Scholes–Merton and put–call parity** are given in doc §6. Do not derive them. Where a formula belongs in your model *is* a design question.

No finance background is assumed. **Out of scope, on purpose:** volatility and interest-rate modeling, auctions and circuit breakers, real market data, concurrency, and persistence.

4 What you have to model

Draw one UML class diagram of **options on a stock**: calls and puts, and the things they need. It must be able to express:

1. **Things that can be owned and valued**, including contracts. A position holds an option in exactly the same way as it holds a share (doc §2, doc §3).
2. **Calls and puts on a stock**, with their terms: underlying, strike, expiry, and multiplier (doc §4). The difference between a call and a put must live in one place.
3. **The two sides of an option**: the holder, who has the right, and the writer, who has the obligation (doc §4.1).
4. **A contract's state** — open, exercised, or expired — and what each state allows (doc §3, doc §5).
5. **Exercise and expiry**: shares and cash moving at the strike, assignment, and automatic exercise at expiry (doc §5).
6. **The value of an option**: its intrinsic value (doc §4.3) and its value before expiry (doc §6).
7. **Construction**: the place where a description from outside your model (a listing, a quoted symbol) first becomes objects. This place is part of your model.

Your design should also make a new kind of option easy to add later, *by adding classes* rather than by changing the classes that already exist. Say how in your rationale.

Two sequence diagrams. Trace each scenario through your class diagram, message by message. Every message must be an operation in your class diagram.

1. At expiry, a call in the money is exercised automatically, and shares and cash change hands (doc §5, doc §8).
2. Someone asks a put for its value before expiry (doc §6).

5 The rule, and how it is marked

Assignment #1 had one rule: boolean logic only inside `AND`, `OR`, and `NAND`. This assignment has a similar rule:

> **No operation in your model may require knowing whether an option is a call or a put, or what kind of asset it is dealing with.**

In code, this rule forbids checks such as `if right == CALL`, and `isinstance` checks on domain types. In a diagram, the same mistake looks like one of these: an operation that only works by listing the kinds it handles; an attribute that another class reads to find out what kind of thing it has; or a note that says "if it is a put, then…". Test every operation you draw with one question: *could this be written without asking what kind of thing it has?*

One exception is allowed: the construction boundary (§4, item 7). It is the one place where your model tells kinds apart, and it does so only once. Name it in your rationale. After construction, everything must work through polymorphism.

How it is marked. Every operation in your diagram is checked with this test. Each failing operation is one violation, and so is each class that exists mainly so that others can ask its kind. The count is reported with your grade.

6 What you submit: Mermaid, and prose

Make a local git repository from the package files. You do not need a GitHub account or a remote. You write no code that simulates the market, in any language. Everything you submit is one of two kinds:

- **Mermaid script — required for every diagram.** Your class diagram is a Mermaid `classDiagram` (`https://mermaid.js.org`), and your two sequence diagrams are `sequenceDiagrams`. Commit them in `design/`, as `.mmd` files or as `mermaid` code blocks in Markdown files. In the class diagram, inheritance ("is-a") and composition ("part-of") must look different; each class shows the attributes that matter and every operation the model needs; show multiplicity where it matters. A diagram that exists only as an image, or in another diagram language, cannot be read and scores zero.
- **Prose, in Markdown — everything else:** `design/rationale.md` (§7); `AI_USAGE.md`, renamed from the template (§8); your marked-up `OPTIONS-MARKETS.md`, at the top of the repository; and a half-page `README.md` that says where things are.

7 The design rationale

`design/rationale.md` is the most important thing you write. Answer these eight questions **for the model you actually drew**, not in general. Say where you used each technique from §2, and why:

1. A share and an option can both be owned, held in a position, and valued (doc §2, doc §3). What do they have in common in your model, and what does that choice cost you elsewhere?
2. Where does the difference between a call and a put belong? Could two parts of your model disagree about what an option is?
3. The holder has a right and the writer has an obligation (doc §4.1). Where do the two sides of one contract live, and how do you keep them consistent?
4. A contract has a life — open, then exercised or expired — but a share does not. Where does that difference belong? What makes it *impossible*, and not just wrong, to exercise an expired contract?

5. Who decides that an option is exercised automatically at expiry: the option, or the rules of the exchange? Why is the other answer wrong?
6. Where does the valuation formula (doc §6) belong, and how does it treat calls and puts differently without asking which one it has?
7. What does an option need to know about its underlying stock, and what should it not know?
8. How would you add a new kind of option later — for example, one that pays a fixed amount — without changing the classes you already have?

Also name **at least two alternatives that you rejected**, and explain why. Finally, state your **misfits**: places where the document describes something that your model does not fit.

8 AI assistant policy

You **may use AI assistants during the take-home, but you must report them**. Keep `AI_USAGE.md` up to date: which tool, when, what you asked for, what it produced, and what you changed or rejected. You are responsible for every box in your diagram, no matter who or what drew it. Not reporting AI use, or not understanding a model that you submitted as your own, is an integrity violation; using the tool is not. Do not share your model or your rationale with other students.

9 The in-class session

You answer **questions** in an online exam form, and at the end you upload an **in-class bundle** (§10). **AI assistants and generative chat tools are off**. You may use your own repository, notes, and diagrams, including `OPTIONS-MARKETS.md`, and one named Mermaid tool, so bring a laptop with your repository on it. Every answer is checked *against your bundles*: Part A against the take-home bundle, and Part B against the in-class bundle. An answer that does not match what you submitted scores zero.

Part A assesses your take-home with ten questions about the model you submitted. Your answers are graded together with your take-home bundle, which scripts and a language model read. Your take-home earns its 75% here, so you must know your own model well. **Part B (25%)** is one design task, planned for two hours, that builds directly on your model. You will not see the task before the in-class session, but you know its shape: like Part B of Asg1, it adds a *dynamic* part of the domain — something that changes over time — that the take-home did not ask you to draw. **You write no code that simulates the market, and no sequence diagram is asked for.** You commit your design to your repository as you work, answer in prose, and type a Mermaid class diagram into an answer box.

10 Submission

Each submission is one text file made by `munger.py`: your commit history, directory tree, and every Markdown and Mermaid file. Run `python3 munger.py` from the top folder of your repository; it asks for your student ID, your name, and **which submission this is**, then writes `<ID>_<Name>_takehome_bundle.txt` or `<ID>_<Name>_inclass_bundle.txt`. Each bundle goes to its own form:

- **Take-home bundle**, before class starts on the day of the in-class session: https://forms.gle/8vRXHktcJs3uJqNG9
- **In-class bundle**, only at the end of the in-class session (one upload per student): https://forms.gle/xXm47BNEg9TRmxi58

Do not modify `munger.py`. Every bundle records the script's SHA-256 digest, which is checked during grading. The published digest is `9bcb352774f0e39237e312eac14cadad6e4bdab08da8cd36c7ed8137fec16c23`. Check your copy with `sha256sum munger.py` (or `shasum -a 256 munger.py` on macOS). Before uploading, check that your work is in the bundle: anything under `FILES NOT BUNDLED`, such as an image-only diagram, is not graded.